Estructural

Creacional

Composite & Factory Method & Builder

OO2 - 2026

Federico Balaguer

Elementos químicos I

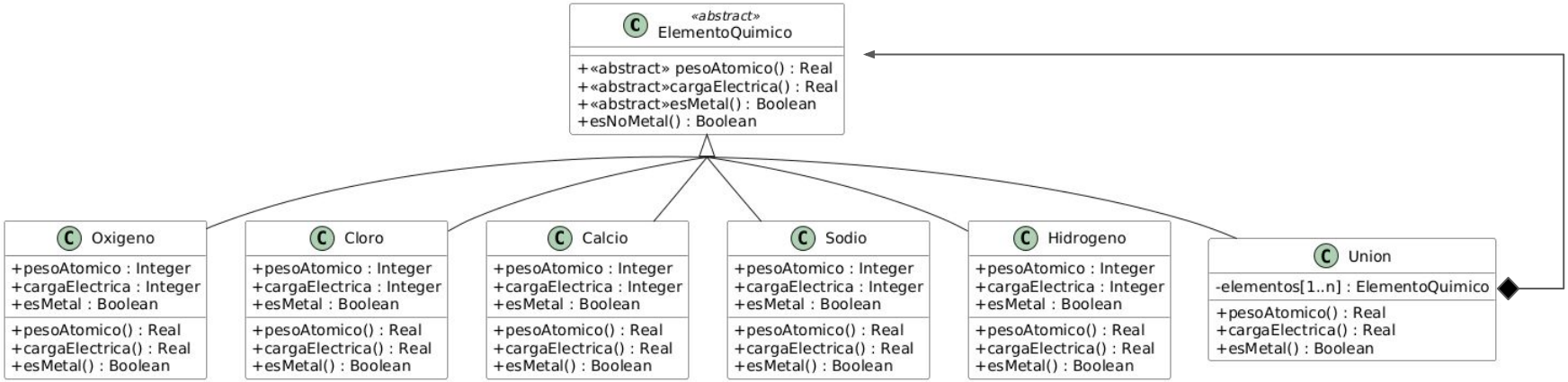
- Elemento
  - Átomos: oxígeno(O), hidrogeno (H), Sodio (Na), calcio (Ca), cloro(Cl)
  - Uniones: $(OH)^-$, H20, NaCl, CaO, NaOH

Símbolo	Peso	Carga	Clasificacion
H	1	+1	No metal
O	16	-2	No metal
Cl	35	-1	No metal
Na	23	+1	Metal
Ca	40	+2	Metal

Reglas	Ejemplo
• Metal + No metal → permitido • No metal + No metal → permitido • Metal + Metal → no permitido	• ❌ Na + Ca → inválido • ✔ Na + Cl → válido (NaCl) • ✔ Ca + O → válido (CaO) • ✔ H + O → válido (H_2O)

Elementos químicos II

- Considerando los uniones:
  - La carga total es la suma de las cargas de sus componentes.
    - Si la carga de la union = 0 → molecula
    - Si la carga de la union != 0 →ión
  - El peso molecular total es la suma de los pesos moleculares de sus componentes.



Variable UnionQuimica.elementos apunta a superclase
ElementoQuimico.esNoMetal() retorna !this.esMetal()

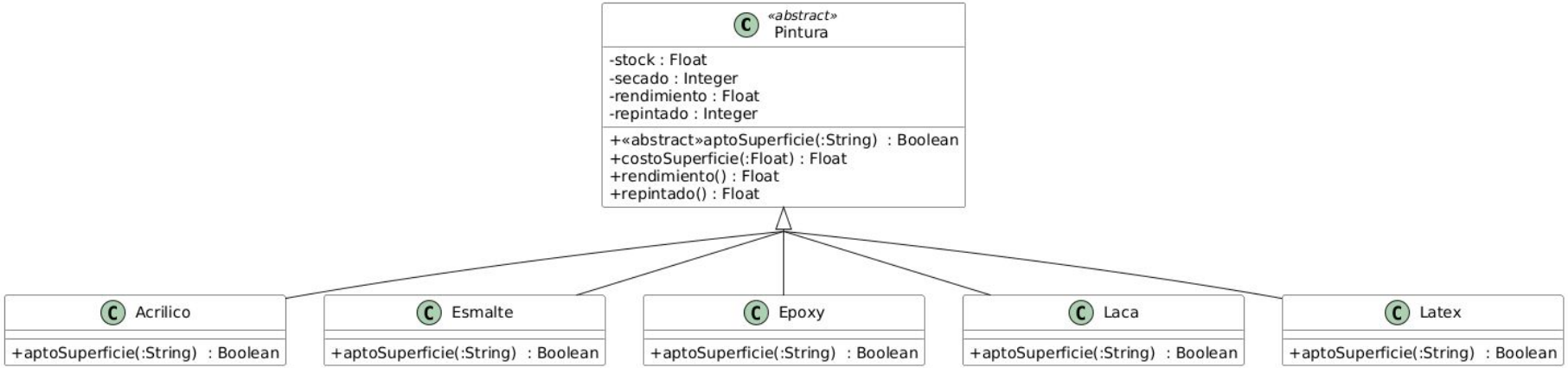
Mezclador de Pinturas

Una empresa constructora utiliza un programa para diseñar pinturas

- Considera colores en stock
- Tipo de pintura requerida (tipo de superficie, tiempo de secado, repintado)
- Superficie a cubrir →Costo
- Requerimiento para transporte (inflamable o no inflamable)
    - Depende de la base o diluyente que usa la pintura

Pintura

- Latex
- Acrilico
- Esmalte
- Laca
- Epoxy



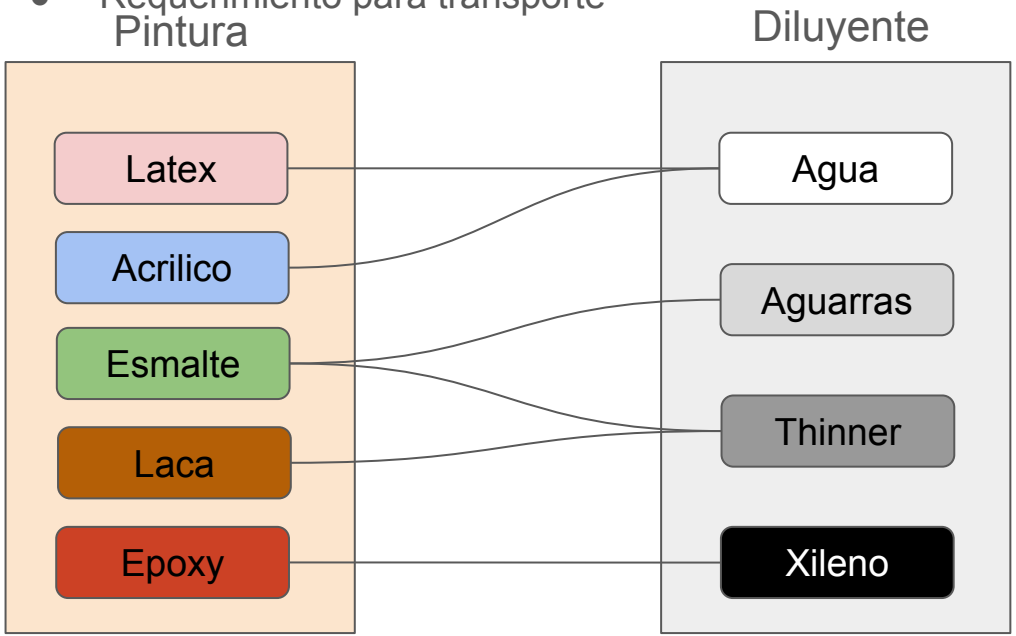
Consideraciones

- El diagrama de clases no incluye constructor que solo setea las variables
- Si la pintura es inflamable o inerte depende de su diluyente
  - Existen pinturas con multiples diluyentes, con diferente indice de inflamabilidad
- El sistema debería permitir agregar con facilidad
  - Nuevos diluyentes
- ⇒ no es buena idea tener la inflamatoriedad de una pintura “hard-coded”
  - Podriamos utilizar el modelo ElementoQuimico para retornar el diluyente
  - Modificar UnionQuimica para que nos diga si es inflamable o no

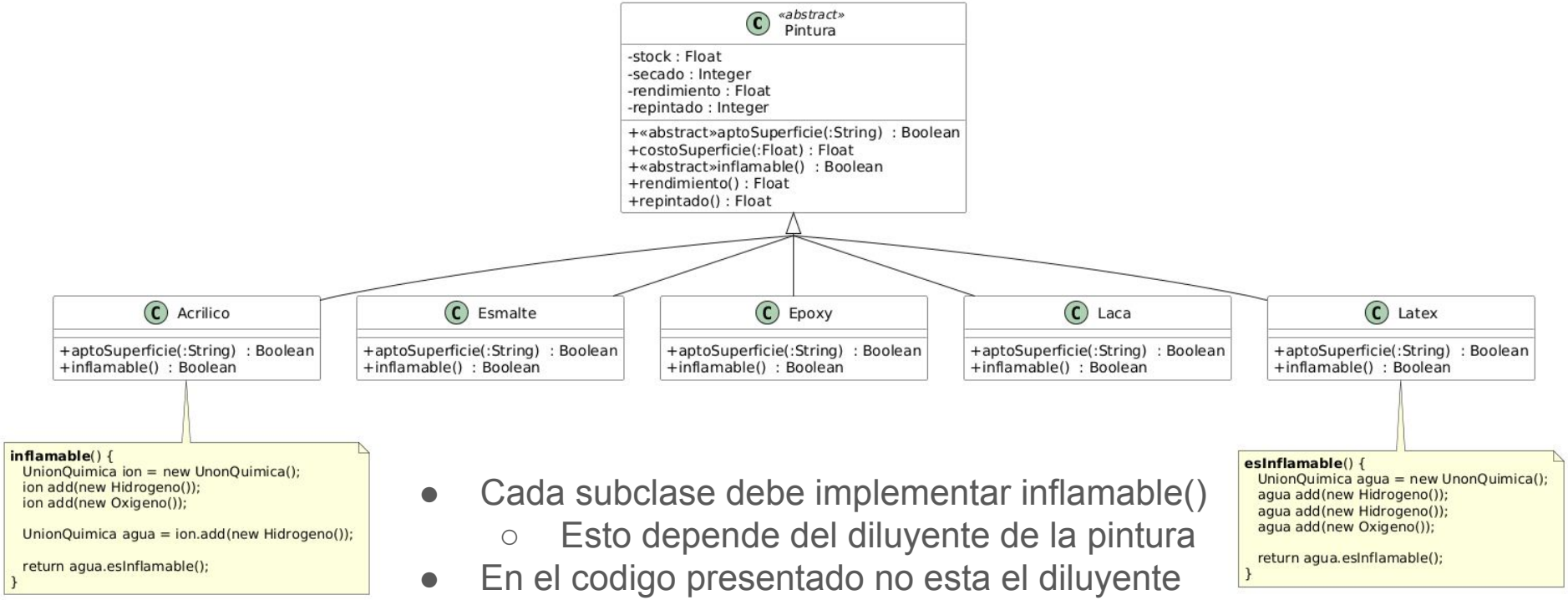
Mezclador de Pinturas

Una empresa constructora utiliza un programa para diseñar pinturas

- Considera colores en stock
- Tipo de pintura requerida (tipo de superficie, tiempo de secado, mantenimiento)
- Superficie a cubrir →Costo
- Requerimiento para transporte



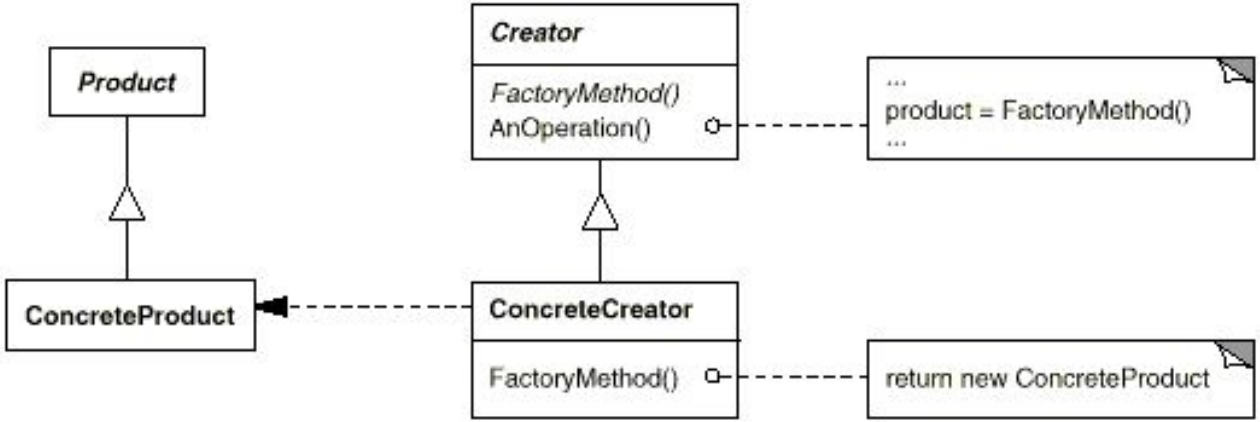
Modelo de Pinturas



- Cada subclase debe implementar inflamable()
  - Esto depende del diluyente de la pintura
- En el codigo presentado no esta el diluyente
  - El diluyente se define con la jerarquía ElementoQuímico

Factory Method

- Intención: Define una “interface” para la creación de objetos, mientras permite que subclases decidan qué clase se debe instanciar. Separa la instanciacion/configuracion de un objeto Producto en un método que puede ser etendido por subclases del creador
- Alias: Virtual Constructor

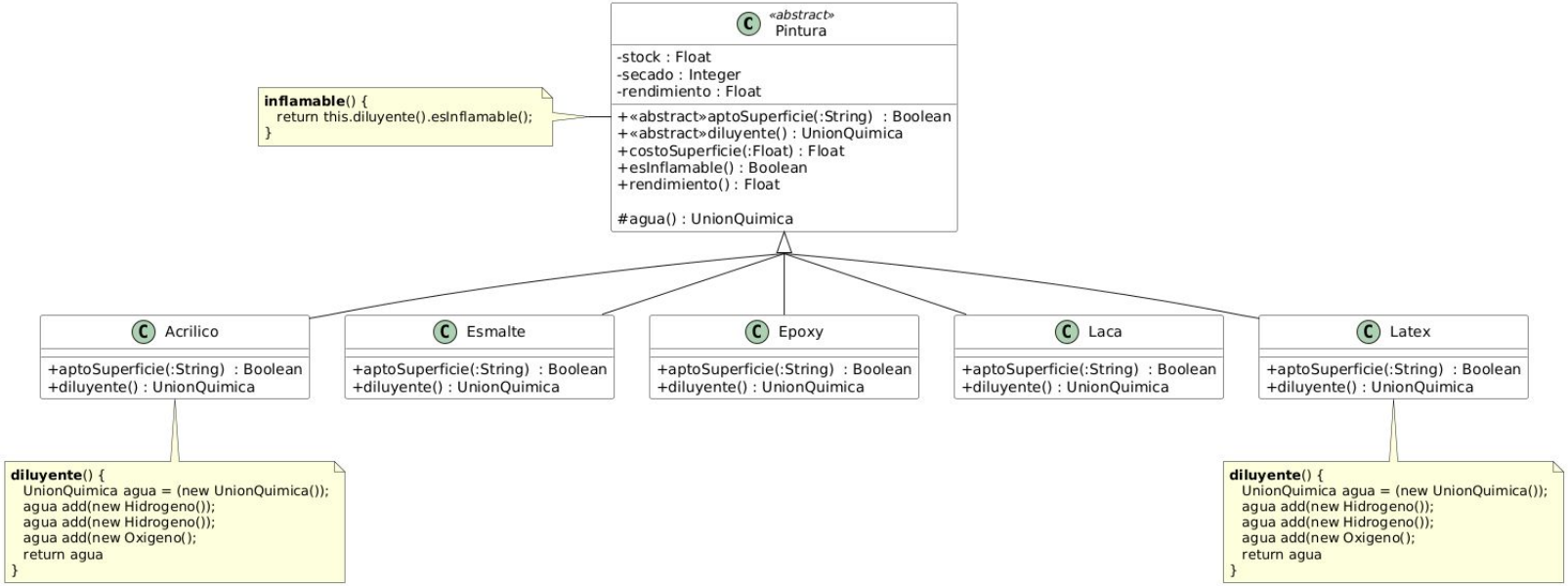


Factory Method

Participantes (Roles)

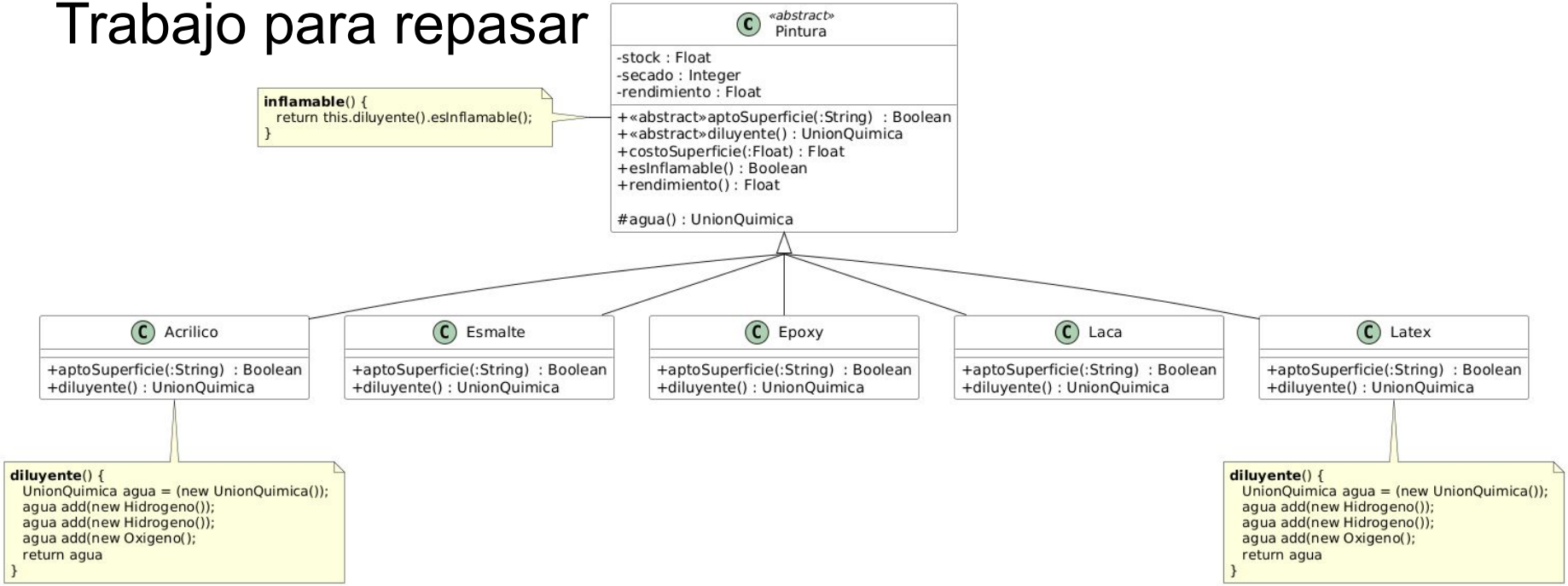
- Product
  - Define la interface of objetos creados por el “factory method”
- Concrete Product
  - Implementa la interface definida por el Product
- Creator
  - Declara el “factory method” (abstracto o con comportamiento default)
- ConcreteCreator
  - Implementa el “factory method”

Diseño resultante I



- inflamable() esta definido en la superclase e invoca a diluyente()
- Diluyente debe ser implementado por todas las subclases
  - Acrilico y Latex tienen el mismo diluyente → código repetido
  - Esto no es un problema de FactoryMethod sino una cuestion del dominio, no invalida la solución propuesta por el patrón

Trabajo para repasar

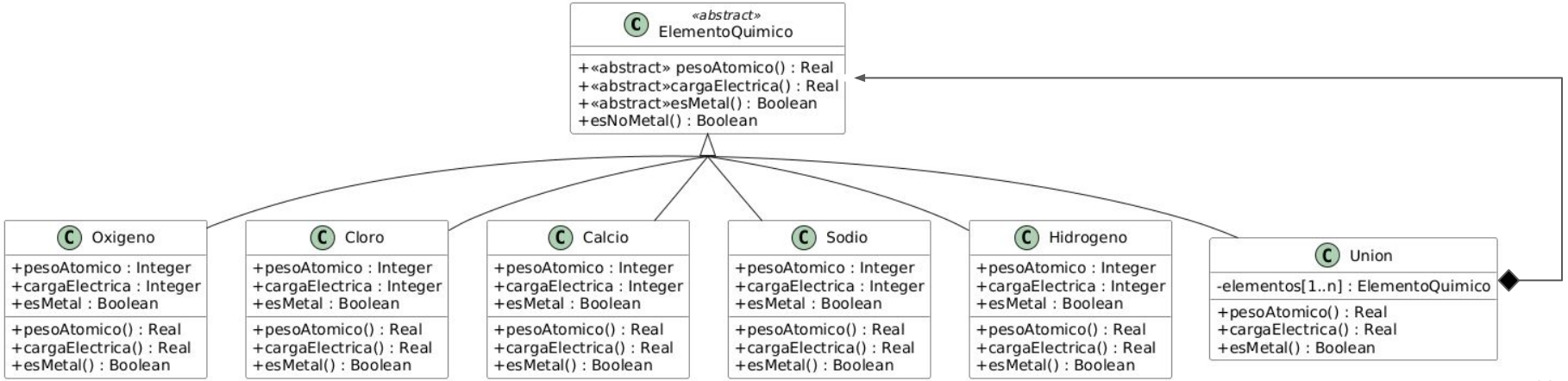


1. Complete el uso del patrón FactoryMethod en el resto de la jerarquía
2. Agregue el metodo *esInflamable()* en la jerarquía de ElementoQuimico
3. Aplique Refactorings vistos en la materia para eliminar el codigo repetido
   a. Cosidere que existe más de una solución
   b. Evalue pros y cons de cada una

Factory Method Unleashed

Factory Method Unleashed I

- Las clases hojas
  - son elementos de la Tabla Periodica de Elementos
  - Tiene los mismas variables y métodos “getters” ⇒ variables y codigo repetido :-(
  - Cada elemento atómico tiene diferentes valores que lo describen
- Por qué no tener una clase única para los elementos atomicos?
  - Quien podría ser <u>responsable</u> de proveer objetos configurados como Oxígeno, Cloro y los otros elementos atómicos

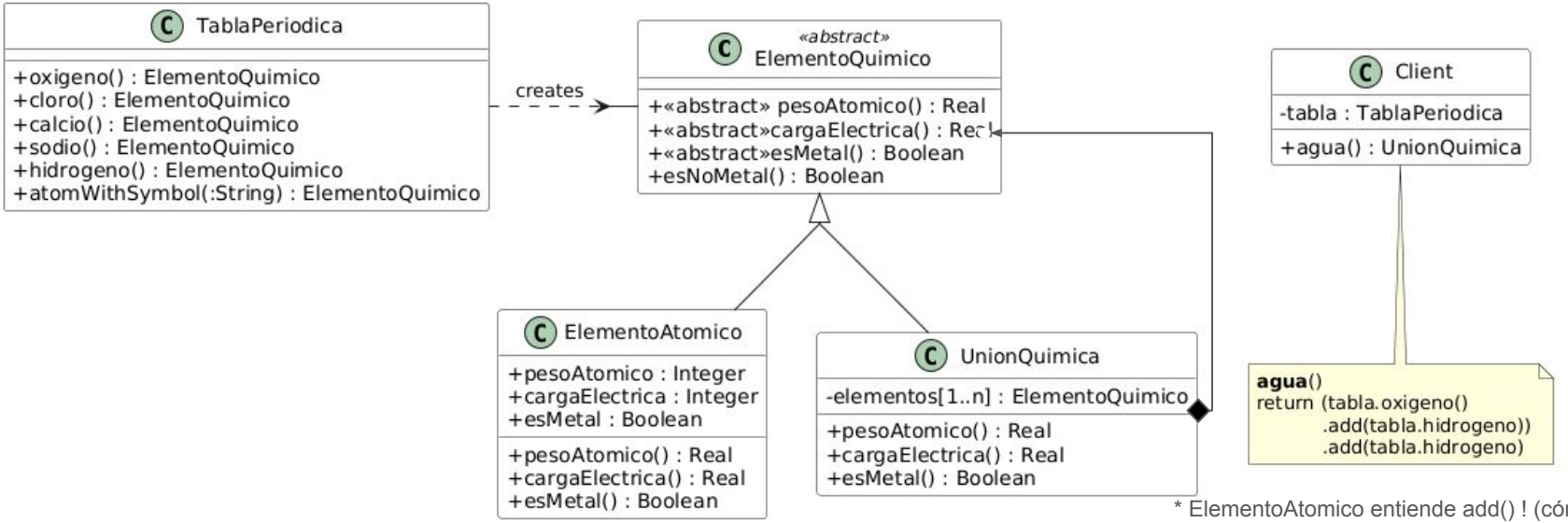


FactoryMethod Unlished II

- Por qué no tener una clase única para los elementos atomicos?
  - Quien podría ser <u>responsable</u> de proveer objetos configurados como Oxígeno, Cloro y los otros elementos atómicos **(responsabilidad == método)**
  - Adonde implementamos el método que configura, por ejemplo, Oxigeno?
    - Podría ser un constructor de ElementoAtómico (sublclase de ElementoQuimico)
      - Hay 118 elementos atomicos en la TablaPeriodica
      - Dependiendo del uso la def de Oxigeno puede cambiar (Simulador, Mezclador, etc)
    - Podría ser un “factory method” de alguien sabe todo sobre Elementos Quimicos Atomicos: TablaPeriodica

FactoryMethod in a nutshell

- Permite codificar la lógica de instanciación/configuracion de objetos en un unico lugar.
  - Nacio como una manera de mitigar el problema de instanciacion con jerarquías paralelas
  - Tiene un valor importante al poner “nombre” a objetos complejos con reglas de instanciacion “fijas”
    - Agua, Agua Oxigenada, SaldeMesa (cloruro de Sodio)...
    - Combo BigMac, Whopper, CeroVeggie…

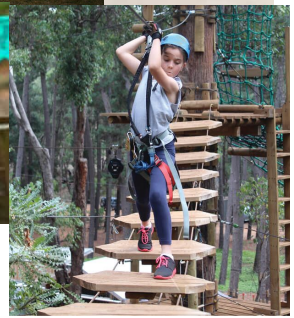


* ElementoAtomico entiende add() ! (cómo?)

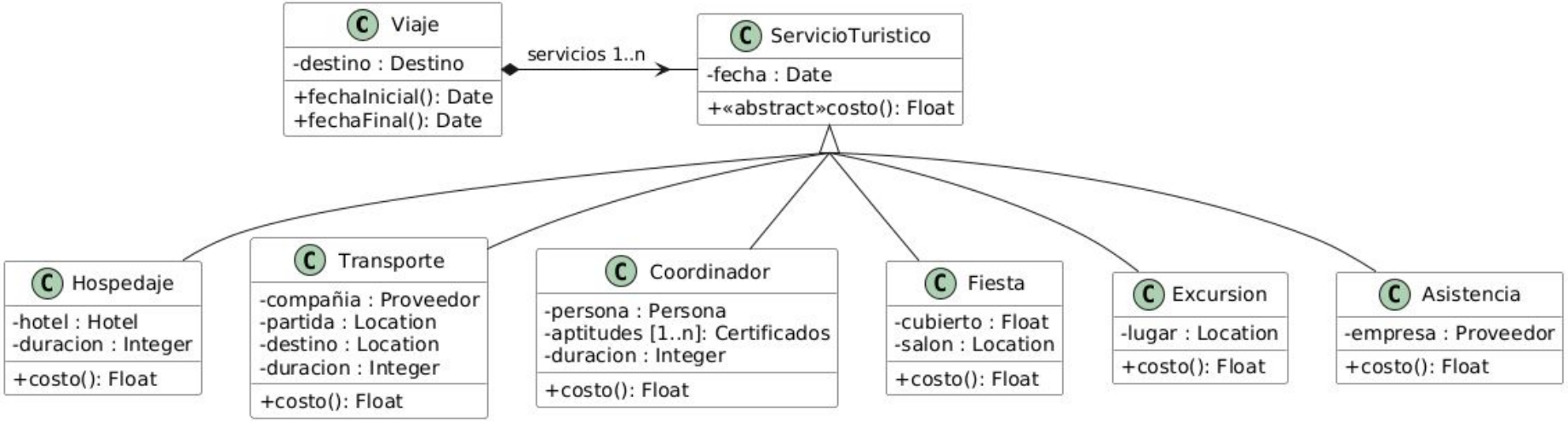
- Transporte
- Alojamiento
- Asistencia Médica
- Coordinación (profes y Guardavidas)
- Fiesta
- Excursiones

El esquema general es el mismo

- Transporte
- Alojamiento c/comidas
- Asistencia Médica
- Coordinación
  - Profes
  - Guardavidas
- Fiesta
- Excursiones



Viajes y sus servicios…



- Cómo se crean los viajes?
  - Las agencias suelen tener diferentes alternativas para cada servicio
    - Economicos
    - Premium
    - No sería una buen diseño tener servicios “duplicados” según sean Premium o Economico

¿Cuál es el problema? (pregunta de final)

- Cambios de estado de un objeto en tiempo de ejecución?
- Una definición única que puede ser “customizable” por subclases?
- Adaptar protocolos entre objetos?
- Manejar la configuración de objetos para que tengan diferente estructura?
  - Pero existe una definición reusable que se aplica a todos los casos
  - La "estructura"
    - se repite en la morfología del producto
    - las partes pueden ser diferentes en cada caso.

Builder

- Intención: separa la construcción de un objeto complejo de su representación (implementación) de tal manera que el mismo proceso puede construir diferente representaciones (implementaciones)
- Estructura (**Roles**)

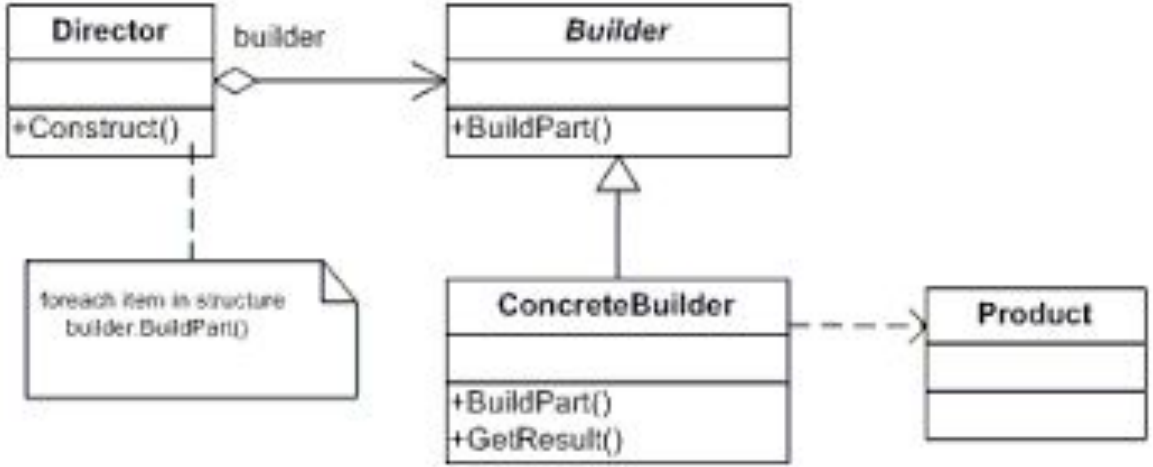
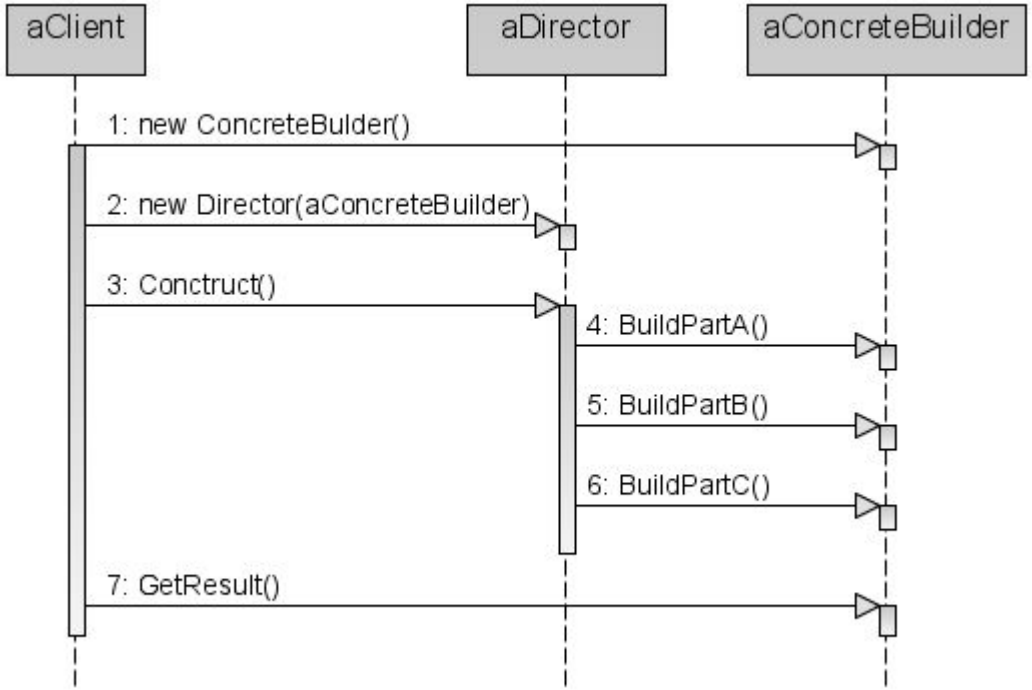


Diagrama de Secuencia



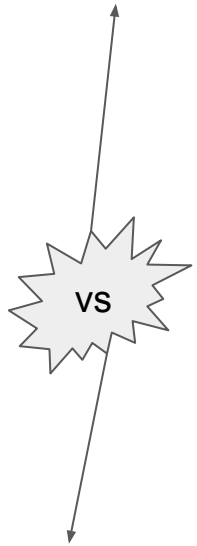
Builder

Participantes

- Builder: especifica una interface abstracta para crear partes de un Producto
- Concrete Builder: construye y ensambla partes del producto.
  - Guarda referencia al producto en construccion
- Director: conoce los pasos para construir el objeto
  - Utiliza el Builder para construir las partes que va ensamblando
  - En lugar de pasos fijos puede seguir una “especificación” (ver knwon uses@ GOF)
- Product: es el objeto complejo a ser construido

Vale la pena Builder?

Uniformidad



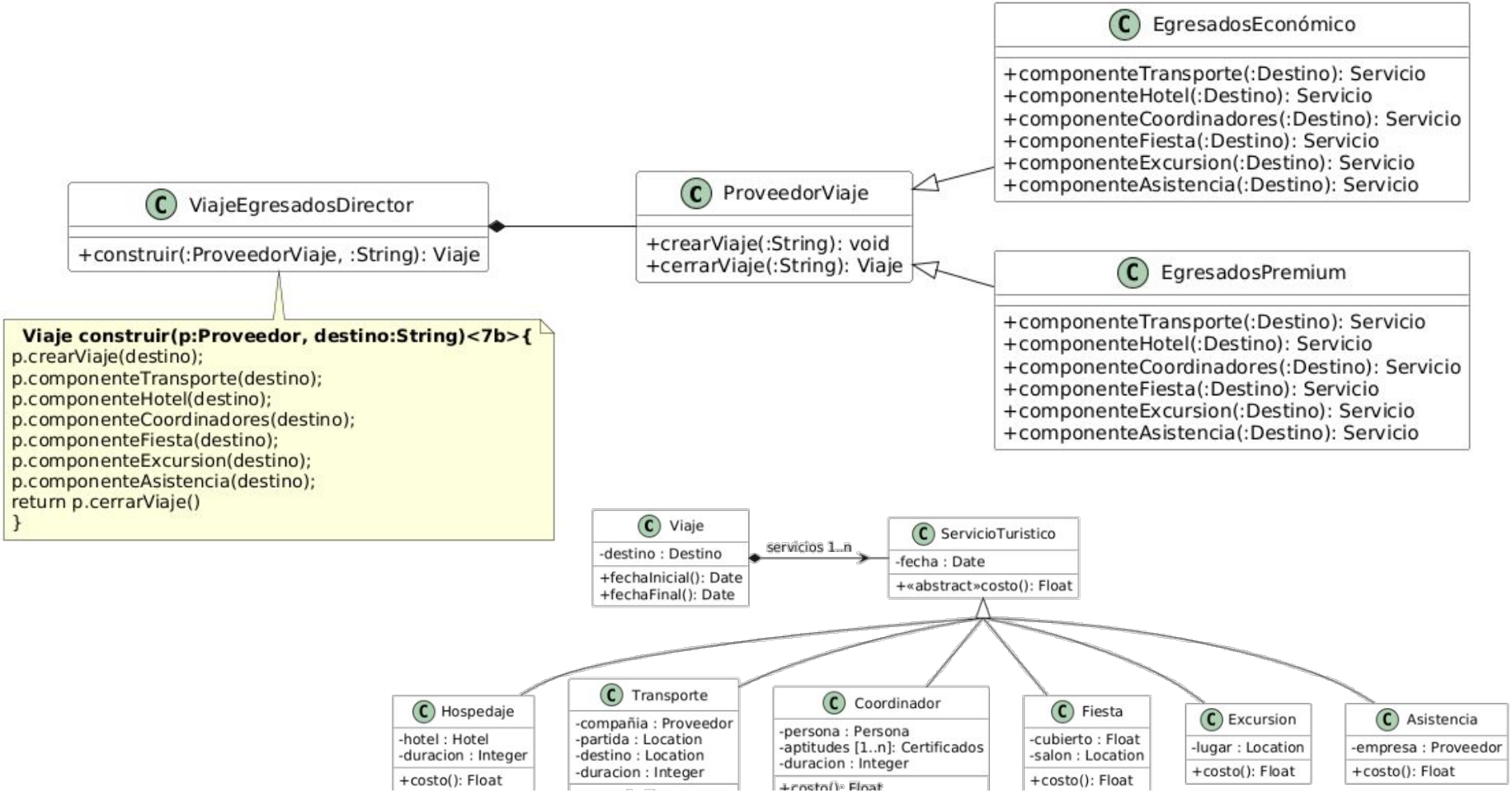
Especificidad

Si!

- Abstrae la construcción compleja de un objeto complejo
- Permite variar lo que se construye Director <-> Builder
- Da control sobre los pasos de contrucción
- Queremos que el “productos”
  - Tengan estructura homogenea
  - Componentes especificos para cada caso

No!

- Requiere diseñar y implementar varios roles
- Cada tipo de producto requiere un ConcreteBuilder
- Builder suelen cambiar o son parsers de specs (> complejidad)


EgresadosEconómico
+componenteTransporte(:Destino): Servicio
+componenteHotel(:Destino): Servicio
+componenteCoordinadores(:Destino): Servicio
+componenteFiesta(:Destino): Servicio
+componenteExcursion(:Destino): Servicio
+componenteAsistencia(:Destino): Servicio
ViajeEgresadosDirector
+construir(:ProveedorViaje, :String): Viaje
ProveedorViaje
+crearViaje(:String): void
+cerrarViaje(:String): Viaje
EgresadosPremium
+componenteTransporte(:Destino): Servicio
+componenteHotel(:Destino): Servicio
+componenteCoordinadores(:Destino): Servicio
+componenteFiesta(:Destino): Servicio
+componenteExcursion(:Destino): Servicio
+componenteAsistencia(:Destino): Servicio
Viaje construir(p:Proveedor, destino:String)<7b>{
p.crearViaje(destino);
p.componenteTransporte(destino);
p.componenteHotel(destino);
p.componenteCoordinadores(destino);
p.componenteFiesta(destino);
p.componenteExcursion(destino);
p.componenteAsistencia(destino);
return p.cerrarViaje()
}
Viaje
-destino : Destino
+fechaInicial(): Date
+fechaFinal(): Date
servicios 1..n
ServicioTuristico
-fecha : Date
+«abstract»costo(): Float
Hospedaje
-hotel : Hotel
-duracion : Integer
+costo(): Float
Transporte
-compañia : Proveedor
-partida : Location
-destino : Location
-duracion : Integer
Coordinador
-persona : Persona
-aptitudes [1..n]: Certificados
-duracion : Integer
Fiesta
-cubierto : Float
-salon : Location
+costo(): Float
Excursion
-lugar : Location
+costo(): Float
Asistencia
-empresa : Proveedor
+costo(): Float

Consideraciones sobre Builder

1. El Director **solo** sabe hacer **una** cosa
2. Los Builders pueden saber hacer cosas que no requiera un Director pero si otro
   a. Ej: componentePsicologico()
3. Los Builders funcionan (generalmente) "injectando dependencias" para armar configuraciones en run-time.
4. Otros Directors pueden usar los mismos Builders.
5. Nuevos servicios ⇒ nuevos Builders
6. Nuevas definiciones de Viajes de Egresado ⇒nuevos directores

Preguntas de Final…

1)¿Cómo se implementa un viaje a Punta del Este?

2) ¿Cómo se implementa un viaje a Río de Janeiro con aéreo y traductor?

3) ¿Cómo se implementa un viaje con múltiples destinos? Ej: Tandil y Miramar

Casos en donde se usa Builder

- Creación en base a una estructura homogenea pero con variantes dependiendo de una condición del dominio
- Casos para estudiar:
  - Seguros de Viajero: estructura común según valor de contratación
  - Pack de seguros: auto, motos, casas según valor de contratación
  - Pack telefonico: celular(gigas, minutos, cantidad de SMS, etc), hogar, TV
  - Servicios en la nube: procesador, memoria, bus de I/O
  - Venta de computadoras on-line…

System	Studio 15	MacBook Pro 15	Studio 17	MacBook Pro 17[1]
Processor	Intel® Core™ i7	Intel® Core™ i7	Intel® Core™ i7	Intel® Core™ i5
Color	Available in 6 colors	Silver	Available in 6 colors	Silver
Webcam	Built-in 2.0MP	Built-in iSight	Built-in 2.0MP	Built-in iSight
Screen	15.6" TrueLife™	15.4" Glossy	17.3" TrueLife™	17" Glossy
Memory	4GB[1]	4GB	4GB[1]	4GB
Hard Drive	500GB[2]	500GB	500GB[2]	500GB
Graphics Mem.	1GB[1]	512MB	1GB[1]	512MB
Software	Windows® Live Essentials	iLife®	Windows® Live Essentials	iLife®
Warranty	1 Year Limited[3] 1Yr LoJack for Laptops Theft Protection	1 Year Limited	1 Year Limited[3] 1Yr LoJack for Laptops Theft Protection	1 Year Limited
Price	$1348.99	$2,199	$1268.99	$2,299
Discount	$399	$0	$219	$0
Total	$949.99	$2,199	$1049.99	$2,299
Savings:	$1249 Buy Now!		$1249 Buy Now!	

Patron	Error
Composite	La jerarquía no es polimorfico
	Composite no tiene relación con otras clases de Jerarquía
	Composite no implementa add/remove
	Composite no delega en la coleccion
Builder	Director construye partes
	Director necesita diferentes builders al mismo tiempo
	Builder manda mensajes a Director
	Solo hay un Builder
FactoryMethod	Jerarquía de Constructores no en el modelo
	Existe una secuencia de FactoryMethods (parametrizados) a ser invocados
	Unico FactoryMethod parametrizado con switch statemen adentro
	FactoryMethod es estático

	FactoryMethod	Builder
Problema	Construcción	Construcción
Encapsula complejidad al cliente?	si/no	si/no
Definición unica ? (receta)	si/no	si/no
Identifica proveedor de partes?	si/no	si/no
Simil constructor?	si/no	si/no
Primer estadío de re-diseño? (usualmente)	si/no	si/no